
A THEORETICAL PERSPECTIVE ON THE FORMAL VERIFICATION OF IOT PROTOCOLS USING LTL AND REWRITING LOGIC IN MAUDE

Delia-Iustina Grigoriță
Faculty of Computer Science
Alexandru Ioan Cuza University
Iași, România
deliaiustinagrigorita@gmail.com

ABSTRACT

As Internet of Things (IoT) systems become increasingly complex, verifying communication protocols is essential to guarantee their security and correctness. This paper offers a brief overview of the theoretical concepts of how to use formal techniques, LTL and rewriting logic within the Maude system to verify the security and proper behavior of the protocols. While rewriting logic explains how various events occur over time, LTL explains how a property can be formulated. This theoretical perspective is intended to inform future applications and research.

Keywords Model Checking, Maude, Rewriting Logic, IoT Protocols, Temporal Logic, Security

1 Introduction

The fast development of the Internet of Things (IoT) has changed the way devices interact and run both in daily life and business surroundings. Comprising linked smart devices, IoT systems run as they should depending on well-defined, safe communication protocols. Maintaining their security and correctness gets more difficult, though, as these systems get more complex [1].

Model checking offers effective methods to discover minor errors in concurrent and reactive systems found in IoT platforms. Turning to formal verification methodologies is important because these approaches ensure strict proof of system quality satisfaction across all possible conditions. Model checking serves as such a verification method by examining all possible states in a system to ensure formal criteria compliance. [1].

To address these challenges, formal verification methods provide rigorous tools for assessing system correctness. Among these, model checking stands out for its ability to automatically and exhaustively explore all possible states of a system in order to verify whether predefined properties hold. This makes it particularly suitable for analyzing the reactive and concurrent behavior often present in IoT environments.

Linear Temporal Logic (LTL) is one of the most widely used logics in this context. It allows the specification of temporal properties, such as guarantees that a condition will eventually be satisfied or that a constraint will always hold—thereby offering a precise means of describing and reasoning about time-dependent system behavior.

Rewriting logic, a computational logic used to specify how systems evolve through transitions, complements LTL by enabling the formal modeling of protocol behavior. Tools such as Maude integrate these two approaches, allowing researchers to define system rules declaratively and reason about their correctness with respect to LTL specifications.

This paper approaches these concepts from a theoretical perspective, aiming to clarify how model checking, LTL, and rewriting logic work together to support the formal verification of IoT communication protocols. Without engaging in implementation details, it outlines the logical foundations of each component and highlights their relevance for the secure design of protocol-driven systems.

The paper is organized as follows: it begins with a section on Model Checking and Transition Systems, followed by an overview of Linear Temporal Logic (LTL). It then presents key properties of IoT protocols and discusses, from a theoretical standpoint, how these protocols may be modeled and analyzed using the rewriting logic framework Maude. The paper concludes with potential directions for future research and a conclusion.

2 The Role of Model Checking and Transition Systems in Formal Verification

Formal verification can test systems such as air traffic control, aircraft separation assurance, autopilots, CPU designs, life-support systems, medical devices (e.g. radiation delivery devices), and many other systems that ensure human safety. It has been used for validation and has been successful.

2.1 What is Model Checking?

Three researchers, Joseph Sifakis along with Ernest Allen Emerson and Edmund Melson Clarke developed model checking during the 1980s. This automated verification system function was developed to verify safety properties in finite models of hardware and software systems. The developed algorithms allowed users to write system properties through formal specifications, which could be checked by model checking programs automatically. When systems were approaching implementation for both hardware products and software development, this method proved essential to locate vulnerabilities and safety issues. Following its initial development, model checking has expanded to address different systems engineering problems, which now include authenticating real-time systems as well as inspecting privacy and security characteristics of computer and communication systems and more. Edmund Melson Clarke, Ernest Allen Emerson and Joseph Sifakis received the Turing Award in 2007 due to their innovative work. Each year the Association for Computing Machinery confers the Turing Award for the most distinguished computer science contributions that have proven their significant impact on the field. The Turing Prize honors Alan Turing by using his name to commemorate the computer science and cryptography pioneer that he was and it serves similar purposes as the Nobel Prize in computer science [2].

There are various criteria for classifying verification approaches, including proof-based or model-based, degree of automation, type of verification (of the entire system behavior or of a specific property), application domain (hardware or software, sequential or concurrent system, reactive or terminating system), and the point at which verification is introduced (before or after system development).

The report resulting from the investigation conducted by the FAA (Federal Aviation Authority) and NASA (National Aeronautics and Space Administration) regarding the use of formal methods concludes that: *"Formal methods should be part of the education of every computer scientist and software engineer, just as the appropriate branch of applied mathematics is a necessary part of the education of all other engineers"* [3] [4].

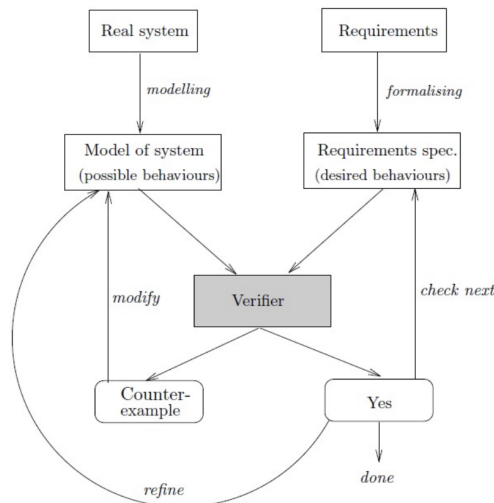


Figure 1: Model checking-verifying a model with respect to specifications [5]

It can be concluded that, in order to perform formal verification, three elements are necessary, as shown in Figure 1: a model of the system, which consists of a set of states containing information about variables and the program state, as well as a transition relation that describes how the system can move from one state to another; a specification method that allows requirements to be expressed in a formal way; and a set of proof rules that help determine whether the model satisfies the specified requirements.

2.2 Fundamentals of Transition Systems

A fundamental concept of describing the behavior of discrete systems is the notion of transition system [6]. It is a set of states and the transitions between the states, where each transition can be labeled. They can be used to label properties of the transition or state, such as the cost or truth value of a condition. If the set of labels consists of a single element, the transition system is unlabeled and can be defined in a simpler way, without the need for labels. Modeling and verification of various types of systems are done using transition systems [6].

The types of protocols we are interested in verifying using LTL can be modeled as transition systems.

Let AP be the (finite) set of atomic propositions, and 2^{AP} the finite alphabet.

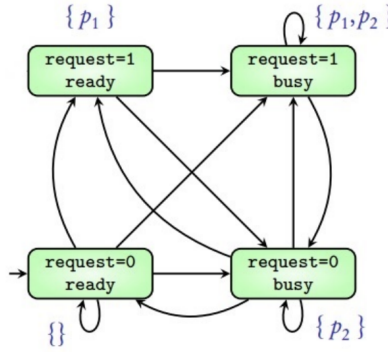


Figure 2: Labeled transition system with four states and atomic propositions [7]

A labeled transition system, such as the one in Figure 2, is a tuple $M = (AP, S, S_0, R, L)$ where:

→ AP is the set of labels (the set of atomic propositions)

→ S is the finite set of states

→ $S_0 \subseteq S$ is the set of initial states

→ $R \subseteq S \times S$ is the transition relation

→ $L : S \rightarrow 2^{AP}$ is the labeling function

The transition system is called finite if both S and AP are finite [7].

A necessary condition for having a transition relation is that each state must have at least one successor. This condition is important in reactive systems, where reactions are dictated by interactions with the external environment. Therefore, it is important that at any moment it is possible to reach another state through the defined transitions. This condition guarantees that there is always an infinite path in the structure defined, because from each state it is possible to reach at least one other state. In such a structure, it is enough to loop the blocking state on itself so that no other state can be reached. Deadlocks can be modeled and issues that could cause system failure can be prevented.

3 Linear Temporal Logic: Foundations, Semantics, and Applications in IoT Protocol Verification

Amir Pnueli introduced Linear Temporal Logic (LTL) for the first time in 1977. The LTL model checking process transforms specifications into Büchi automata through automata-theoretic methods. The automaton operates as a finite automaton, which handles infinite words to accept computations that fulfill the specified formula [8].

Linear Temporal Logic is used in model checking to specify certain properties; in the case addressed in this work, it is used to specify certain security properties to determine whether the Internet of Things protocol, which we have modeled as a transition system, is secure or not according to the specification given in LTL.

There are two distinct natures of temporal logic: linear and branching. In a linear perspective, there is only one successor at any given moment in time, while in a branching perspective, multiple alternative paths may exist [9]. This work focuses on the linear perspective of temporal logic, based on the formalism of Linear Temporal Logic.

LTL is a modal logic that refers to time and differs from classical logic in that formula evaluation takes place over a set of states, not just in a single fixed state. In classical logic, a formula must be true or false in one state, but in temporal logic, the truth of a formula can vary depending on the state in which it is evaluated. Therefore, a proposition like "it is raining" can be true in some states but false in others.

3.1 Modal logic

Modal logic emerged in the '60s as a subcategory of formal logic, extending classical propositional and predicate logic by including operators expressing modality. Modals express modalities through words that qualify statements by adding adverbs or adverbial phrases. [10].

A formal modal logic is used to represent modalities through the use of modal operators, such as "Possible" and "Necessary." These operators are used to express notions like the possibility or necessity of a statement. For example, "It might rain today" and "It is possible that it will rain today" both express the idea of possibility. In modal logic, this is represented by the "Possible" operator applied to the statement "It will rain today." It is important not to confuse the notion of necessity with that of possibility. Confusing these is known as the modal fallacy.

The basic unary modal operators (with a single argument) are often denoted by $\Box$ for "Necessary" and $\Diamond$ for "Possible". Using the previous example, if P represents the statement "it will rain today", the possibility of rain can be represented by $\Diamond P$, which means "It is possible that it will rain today". Similarly, $\Box P$ can be used to express "It is necessary that it will rain today", suggesting the certainty of the statement [10].

3.2 LTL syntax

LTL formulae are based on a few fundamental elements, including atomic propositions, as well as Boolean logical connectors such as conjunction ($\wedge$) and negation ($\neg$). LTL also uses two essential temporal modalities: "next" and "until", to express relationships between events and points in time [3].

LTL formulae over the set AP of atomic propositions are defined according to the following Backus–Naur grammar [3]:

$$\varphi ::= \text{true} \mid a \mid \varphi_1 \wedge \varphi_2 \mid \neg \varphi \mid \bigcirc \varphi \mid \varphi_1 \cup \varphi_2$$

A finite set of atomic propositions: the state label " a " $\in$ AP in the transition system. In addition to the usual logical operators, add the following temporal operators:

- X (next) or $\bigcirc$, $X\varphi$
- F (eventually) or $\Diamond$, $F\varphi$
- G (always) or $\Box$, $G\varphi$
- U (until), $\varphi U \varphi$
- W (weak until), $\varphi W \varphi$
- R (release), $\varphi R \varphi$

3.3 LTL semantic

An LTL formula defines a certain property, and its semantics are defined through a language called "Words", which contains all infinite words over a given alphabet that satisfy that property. Subsequently, the semantics are extended to an interpretation of paths and states within a transition system [3].

The protocols we want to verify must be modeled as Kripke transition systems (models), so let $M = (S, \rightarrow, L)$ be a model and let $\pi = (s_1 \rightarrow s_2 \rightarrow \dots)$ be a path in M . Whether it π satisfies an LTL formula is defined by the satisfiability relation $\models$ as follows:

1. $\pi \models \top$,
2. $\pi \not\models \perp$,
3. $\pi \models p$ iff $p \in L(s_1)$,
4. $\pi \models \neg\varphi$ iff $\pi \not\models \varphi$,
5. $\pi \models \varphi_1 \wedge \varphi_2$ iff $\pi \models \varphi_1$ and $\pi \models \varphi_2$,
6. $\pi \models \varphi_1 \vee \varphi_2$ iff $\pi \models \varphi_1$ or $\pi \models \varphi_2$,
7. $\pi \models \varphi_1 \rightarrow \varphi_2$ iff $\pi \models \varphi_2$ whenever $\pi \models \varphi_1$,
8. $\pi \models X\varphi$ iff $\pi^2 \models \varphi$,
9. $\pi \models G\varphi$ iff for all $i \geq 1$, $\pi^i \models \varphi$,
10. $\pi \models F\varphi$ iff for some $i \geq 1$, $\pi^i \models \varphi$,
11. $\pi \models \varphi_1 U \varphi_2$ iff $\exists i \geq 0$, $\pi^i \models \varphi_2$ and for all $j < i$, $\pi^j \models \varphi_1$,
12. $\pi \models \varphi_1 W \varphi_2$ iff $\exists i \geq 0$, $\pi^i \models \varphi_2$ and for all $j < i$, $\pi^j \models \varphi_1$ or for all i , $\pi^i \models \varphi_1$,
13. $\pi \models \varphi_1 R \varphi_2$ iff $\exists i \geq 0$, $\pi^i \models \varphi_1$ and for all $j < i$, $\pi^j \models \varphi_2$ or for all i , $\pi^i \models \varphi_2$.

A path in the model $M = (S, \rightarrow, L)$ is an infinite sequence of states $s_1, s_2, s_3, \dots$ from S , such that for any $i \geq 1$, we have $s_i \rightarrow s_{i+1}$ [7]. This path can be written as $s_1 \rightarrow s_2 \rightarrow \dots$.

If we consider the path $\pi = s_1 \rightarrow s_2 \rightarrow \dots$, it represents a possible future of the system: starting from state s_1 and continuing with state s_2 , and so on [11].

It's denote by π^i the suffix starting with state s_i [11].

The first and second clauses reflect the fact that an atomic formula is always true, while a contradictory formula is always false.

Clauses 3–7 are similar to the corresponding ones from propositional logic.

Clause 8 removes the first state from the path to create a new path starting from the second state.

Clauses 9–13 involve the logical operators extended in temporal logic (3.2). Also, the future includes the present. This means that when we say "in all future states," we also include the present state as a future state.

As we can see in Figure 3, the "Next" operator specifies that a certain property "a" must be true in the next state of the system.

The "Future" ("Eventually") operator specifies that property "a" must become true at some point along a future path of the system.

The "Globally" ("Always") operator specifies that property "a" must be true throughout the entire path of the system.

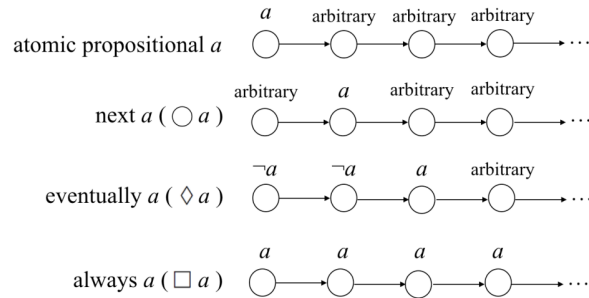


Figure 3: An example of the semantics of temporal operators [12]

What is not shown in Figure 3 is that the "Until" operator specifies that a certain property "a" must be true now or in the future, and another property "b" must be true until that moment. After that moment, "a" may no longer need to be true.

There are two other operators in temporal logic: W, called "Weak-until", and R, called "Release". The Weak-until (W) connective is similar to Until (U), except that "a W b" does not require "b" to eventually be satisfied along the path, as is required in "a U b". The Release (R) connective is the dual of Until (U); that is, "a R b" is equivalent to " $\neg(\neg a U \neg b)$ ". The name "Release" comes from the fact that clause 11 states that "b" must remain true up to and including the point where "a" becomes true. The connectives R and W are quite similar, with the difference being that they swap the roles of "a" and "b", and that the clause for W has an $i - 1$ where R has i . Although the two connectives are similar and can be defined in terms of each other, it is useful to have both. The R connective is useful because it is the dual of U, and W is useful because it is a weaker form of U. It is important to note that neither the strong variant (U) nor the weak variant (W) of Until says anything about what happens after the until condition has been fulfilled [3].

3.4 Ensuring Correctness in IoT Protocols Through LTL Verification

LTL can be used to define the properties that IoT protocols must satisfy in order to be considered correct. For example, you can define the property that an IoT device must receive a confirmation message before sending another message or that all devices must be connected to the network before starting communication.

Once the properties are defined, LTL can be used to verify whether the IoT protocols comply with those properties. The verification can be performed using an LTL model checker, which automatically checks whether the protocols conform to the defined properties.

The use of LTL for modeling and verification of IoT protocols leads to correct communication and decreases the probability of device errors or conflicts.

The process of creating an LTL specification for an IoT protocol requires defining the desired system properties and behaviors. The properties exist in two forms: natural language statements and LTL formula sets. These are some typical properties that IoT protocols usually possess:

Authentication: "Any device that connects to the IoT network must be authenticated before it can communicate with other devices." This can be expressed in LTL as: $\mathbf{G}(\text{connected} \rightarrow \text{authenticated})$, where *connected* and *authenticated* are atomic propositions representing the connection and authentication states of a device.

Confidentiality: "Data transmitted between IoT devices must be encrypted to protect sensitive information." This can be expressed in LTL as: $\mathbf{G}(\text{transmitting} \rightarrow \text{encrypted})$, where *transmitting* and *encrypted* are atomic propositions representing the state of data transmission and encryption.

Integrity: "Data transmitted between IoT devices must not be altered during transfer." This can be expressed in LTL as: $\mathbf{G}(\text{transmitting} \rightarrow \neg \text{modified})$, where *modified* is an atomic proposition representing the state of data being altered.

Availability: "Any IoT device must remain available for communication at all times." This can be expressed in LTL as: $\mathbf{G}(\text{available})$, where *available* is an atomic proposition representing the availability state of a device.

These are just a few examples of properties you can specify using LTL for an IoT protocol. It is important to identify and model all necessary behaviors and properties to ensure the correct and secure operation of the system.

4 Analysis of IoT Protocols

4.1 IoT Protocols Properties and Their Importance

IoT protocols consist of a set of standardized rules and conventions through which Internet-connected devices can communicate with each other and also interact with one another.

IoT protocols are important because they ensure the following properties:

- **Security:** Internet connected devices often transmit and store sensitive information, meaning confidential and private data. Therefore, IoT protocols protect our data through security measures such as data encryption, authentication of the devices involved in communication, and authorization to allow only appropriate access to resources. The same IoT protocols can also prevent attacks by implementing detection mechanisms, for example, digital signatures or two-factor authentication.
- **Efficient Communication:** Communication between devices is optimized through IoT protocols, thus bandwidth is used efficiently (the amount of information that can be transmitted within a specific time interval), which is

important for fast and efficient communication, especially since many IoT devices have low power or unstable connections. One of the methods to efficiently manage bandwidth is data compression, where redundant data is eliminated to reduce the size of data packets.

- **Scalability:** This is important for IoT protocols because it facilitates connectivity and supports a large number of devices within a bigger, more extensive network. Therefore, these protocols must be able to handle the increasing number of devices (sensors, smart appliances, etc.), which can reach hundreds or thousands. Moreover, scalable networks have other notable properties such as: the ability to support a high volume of data, to accommodate new functionalities, to integrate new devices, to adapt to the evolution of IoT technology, to efficiently manage energy consumption, and to ensure connectivity between devices located in different geographical areas, etc.
- **Standardization:** Provides a common and coherent framework for communication and interaction between IoT devices and applications. Standardization enables simpler development along with implementation and integration of IoT solutions because manufacturers can utilize standardized protocols for achieving interoperability and compatibility between different devices and platforms. A defined set of common rules together with accepted protocols and specifications needs to be established.
- **Compatibility:** Refers to the ability of IoT devices to communicate in a seamless and problem-free manner, meaning they must use the same protocol and communicate and interact with each other through that same protocol. Therefore, this facilitates the integration of IoT devices into a system or network and ensures that users can enjoy the benefits of IoT technology without being limited by compatibility constraints.
- **Interoperability:** This property is essential to allow devices to communicate efficiently and without issues or difficulties. Since the IoT market is highly diverse and there are many manufacturers of various devices, it is very important that these different devices can communicate within a network together in a standardized way, without the need for modifications. Interoperability also implies flexibility and scalability, data being transmitted and interpreted correctly, collaboration between different providers, and the facilitation of IoT system expansion across various industries and sectors.
- **Network and Device Management:** This property enables the configuration and monitoring of devices, including changing operating parameters and settings. IoT protocols include mechanisms for problem diagnosis and device and network troubleshooting, allowing us to determine the status of devices, communication errors (if any), or battery levels. Other mechanisms ensure firmware and software updates for devices, thus facilitating the administration and implementation of a highly complex IoT network.

4.2 Overview of IoT Protocols Suitable for Model Checking

There are several protocols that can be verified through model checking, including:

- **MQTT** – Message Queuing Telemetry Transport is a publish/subscribe messaging protocol, meaning that devices can publish and receive messages on specific topics through a central MQTT broker (which sends messages to the devices subscribed to various topics) [13]. This protocol is designed to be simple, easy to implement, and easy to use, making it ideal for use in constrained environments [14].
- **CoAP** – Constrained Application Protocol is a network protocol optimized for resource-constrained devices and networks. It is similar to the HTTP protocol as it features a request-response architecture. Communication is carried out using the UDP (User Datagram Protocol); however, there is indeed a drawback: using this protocol, it cannot guarantee the delivery and order of messages. On the other hand, we can say the following: communication is easier to establish and faster compared to TCP.
- **RADIUS** – Remote Authentication Dial-In User Service [15] is an authentication, authorization, and accounting protocol used in communication networks to manage user access. It follows a client/server model. The RADIUS client is generally a daemon process (i.e., it runs in the background, has no graphical interface, provides services or functionalities in the background, and runs independently, meaning the user does not need to intervene directly). The client sends the user's information to the RADIUS servers and acts based on the received response. In contrast, the RADIUS servers receive the user's connection requests, authenticate the user, and return the necessary configuration data for the client to provide services to the user.
- **Zigbee** is a wireless communication protocol, meaning it operates without cables, developed by the HomeRF Alliance and currently managed by the ZigBee Alliance. Its development began in 1997 and it is based on the IEEE 802.15.4 standard (which defines the physical and media access control specifications). This protocol stands out due to several positive aspects, such as low power consumption – a very important feature in daily use – low bandwidth, flexibility, and it provides security functionalities such as encryption and authentication.

- Z-Wave functions as a wireless communication protocol which enables smart devices to establish connections for exchanging control data and commands. Z-Wave devices function with minimal power usage which enables them to maintain battery power for extended durations. The system enables central management through hubs or gateways, which let users control and monitor Z-Wave devices using different applications and web interfaces.
- Thread is a wireless network protocol for connectivity and communication. It features a mesh topology, offers low power consumption, and ensures security and interoperability. It uses IPv6 for network connectivity, which means more IP addresses are available, and it facilitates integration with the existing Internet infrastructure.

Modeling and verifying IoT protocols using linear logic is the process of creating a formal model of a protocol and verifying that the model satisfies a set of desired properties expressed in LTL specifications.

The first step in this process is defining the protocol and specifying the behaviors and properties to be verified. This may include specifying the sequence of events that occur during the execution of the protocol, the messages exchanged between devices, and the desired properties, such as the security of the protocol.

Once the model has been created, the next step is to specify the desired characteristics of the protocol using LTL expressions. Finally, model checking tools are used to verify whether the model satisfies the desired properties expressed in the LTL formula. Model checking is an automated technique used to verify whether a formal model of a system satisfies a set of desired properties. A model checking tool can analyze the behavior of a model and verify whether it meets the desired properties or generate a counterexample otherwise (e.g., SPIN, the Maude LTL logical model checker (LMC)) [16].

4.3 Maude: A Framework for Rewriting Logic

Maude is a high-performance programming language that supports both executable specifications and declarative programming with rewriting logic. Since rewriting logic is a type of equality logic, Maude can support both specification-based and equality-based programming by using modular sublanguages and function theory. Maude uses the reflectivity of rewriting logic to represent meta-levels at the object level and offers powerful meta-programming capabilities, such as user-defined module operations and declarative strategies to guide the reasoning process [17] [18].

Rewriting logic is a simple computational logic that can express both concurrent computation and logical inference in a very general way. Over the past two decades, numerous researchers have developed the foundations, semantic and logical frameworks, language implementations, and formal tools of rewriting logic, also working in the fields of automated inference, software and hardware specification and verification, security, real-time cyber-physical systems, stochastic systems, bioinformatics, chemical systems, and many other areas of application [19].

4.4 Formal Modeling

Defining an IoT protocol using rewriting logic means, firstly, modeling the protocol syntax and, secondly, semantics through rewriting logic. Generally, this is achieved by listing what the different kinds of messages and states a protocol may have, and how the messages and states can transition between them [20] [21].

Maude also allows defining protocols behavior in a more modular and declarative way. A protocol can be created with the different message routing modules, state transition modules, and error handling modules separately and then combined.

Then if you model it in Maude you can use the verification and analysis tools inside of Maude to just verify that it meets the specifications to verify that it's not vulnerable to attacks [22] [23].

4.5 Model Checking in Maude Using LTL

The LTL syntax in Maude is the following functional LTL module (defined in the file model-checker.maude) [24] [22].

In order to perform a verification on what is modeled in Maude (see Fig. 4 from Appendix), is needed to define more precisely what we want to verify.

In Maude, the process of verifying whether an LTL formula is successfully satisfied by the model is done using the **search** command. The syntax of the **search** command is designed to explore the space of reachable states, following a breadth-first strategy through various exploration methods. This syntax follows a specific general pattern [17]:

```
search [ n, m ] in <ModId> :
```


$\langle \text{Term-1} \rangle \langle \text{SearchArrow} \rangle \langle \text{Term-2} \rangle$
such that $\langle \text{Condition} \rangle$.

Where each represents the following [17]:

- an optional argument n , which specifies the maximum number of desired solutions,
- an optional argument m , which specifies the maximum search depth,
- the module $\langle \text{Module} \rangle$ performing the search can be omitted,
- the initial term is indicated by $\langle \text{Term-1} \rangle$,
- and the pattern to be reached by $\langle \text{Term-2} \rangle$,
- the arrows indicate the form of the rewriting relation from $\langle \text{Term-1} \rangle$ to $\langle \text{Term-2} \rangle$,
- $\Rightarrow 1$ is a rewriting relation consisting of a single step,
- $\Rightarrow +$ is a rewriting relation consisting of one or more steps,
- $\Rightarrow *$ is a rewriting relation consisting of zero, one, or more steps. Zero indicates a rewriting relation consisting of one or more steps.
- $\Rightarrow !$ indicates that no further rewrites are allowed, only the final normal form state.

It is also possible to specify optional properties that the state reached by $\langle \text{condition} \rangle$ must satisfy, which have the same syntactic form as the condition of a conditional rule.

5 Future Directions

While this paper focused on the theoretical aspects of using LTL and rewriting logic for the formal verification of IoT protocols, several avenues remain open for further exploration. A natural next step is the implementation of concrete case studies, where real-world IoT protocols (e.g., MQTT, CoAP, Zigbee) are modeled in Maude and verified against relevant temporal logic specifications.

Another direction lies in the integration of probabilistic and real-time extensions into the verification process. Given the dynamic and often unpredictable nature of IoT environments, incorporating probabilistic reasoning or real-time constraints into LTL specifications and rewriting logic models could significantly enhance the realism and relevance of the verification outcomes.

Moreover, there is increasing interest in combining formal verification with machine learning techniques. For example, anomaly detection algorithms could be used to dynamically generate LTL properties from observed system behaviors, which are then formally verified using model checking.

6 Conclusion

Making sure IoT systems operate properly is more important than ever as they become increasingly common in our daily lives. In this work, we present how formal approaches, rewriting logic and model checking with Linear Temporal Logic (LTL) using Maude, can aid in understanding and validation of the behavior of Internet of Things communication protocols. The aim was to introduce the conceptual foundations of these tools and show how they are interconnected rather than concentrating on implementation. While rewriting logic allows us to model how a system changes over time, LTL allows us to describe what should happen in a system over time. Together, they provide a potent means of verifying that a protocol performs as expected.

Using model checking techniques, problems in IoT protocols that might undermine the robustness and trustworthiness of IoT systems can be discovered and addressed before their deployment. LTL-based model checking has been shown to be a viable and effective technique for verifications of some essential properties of IoT communication protocols.

Model verification is a complex and resource-consuming process that requires great research effort and methodological rigor. Nevertheless, when correctly applied, it presents strong assurances of the fact that IoT protocols meet the agreed-upon criterion of security and correctness, alleviating future risks.

References

- [1] W. Ji, J. Xu, H. Qiao, M. Zhou, and B. Liang, “Visual iot: Enabling internet of things visualization in smart cities,” *IEEE Network*, vol. 33, no. 2, pp. 102–110, 2019.

- [2] E. A. Emerson, “The beginning of model checking: A personal perspective,” in *25 Years of Model Checking: History, Achievements, Perspectives*. Springer, 2008, pp. 27–45.
- [3] C. Baier and J.-P. Katoen, *Principles of model checking*. MIT press, 2008.
- [4] M. Penicka, “Towards a theory of railways,” Ph.D. dissertation, Citeseer, 2006.
- [5] D. Fötsch, A. Speck, W. R. Rossak, and J. Krumbiegel, “A concept for modelling and validation of web based presentation templates,” in *Wirtschaftsinformatik 2005: eEconomy, eGovernment, eSociety*. Springer, 2005, pp. 391–406.
- [6] D. Raffae, “The concept of transition system,” *Journal of education and work*, vol. 21, no. 4, pp. 277–296, 2008.
- [7] M. Huth and M. Ryan, *Logic in Computer Science: Modelling and reasoning about systems*. Cambridge university press, 2004.
- [8] K. Y. Rozier, “Linear temporal logic symbolic model checking,” *Computer Science Review*, vol. 5, no. 2, pp. 163–203, 2011.
- [9] E. A. Emerson and J. Y. Halpern, ““sometimes” and “not never” revisited: on branching versus linear time temporal logic,” *Journal of the ACM (JACM)*, vol. 33, no. 1, pp. 151–178, 1986.
- [10] A. Chagrov and M. Zakharyashev, *Modal logic*. Oxford University Press, 1997.
- [11] F. Fioravanti, A. Pettorossi, M. Proietti, and V. Senni, “Program transformation for development, verification, and synthesis of programs,” *Intelligenza Artificiale*, vol. 5, no. 1, pp. 119–125, 2011.
- [12] B. Srongsil and W. Vatanawood, “Compositional verification of data invariants in promela using slicing technique,” in *Proc. Int. MultiConf. Eng. Comput. Scientists*, 2018, pp. 484–488.
- [13] A. Shaikh, “A survey on exchanging data using mqtt protocol in arduino,” *Int J Res Appl Sci Eng Technol*, vol. 9, pp. 3081–3082, 2021.
- [14] K. M. Alam and A. Akram, “A survey on mqtt protocol for the internet of things,” *Khulna University, Dept. of Computer Science and Engineering (CSE)*, 2016.
- [15] J. Hill, “An analysis of the radius authentication protocol,” 2001.
- [16] A. Saraswat, K. Abhishek, M. R. Ghalib, A. Shankar, M. Alazab, and B. Nongpoh, “Towards energy efficient approx cache-coherence protocol verified using model checker,” *Computers & Electrical Engineering*, vol. 97, p. 107482, 2022.
- [17] M. Clavel, F. Durán, S. Eker, P. Lincoln, N. Martí-Oliet, J. Meseguer, and J. F. Quesada, “Maude: Specification and programming in rewriting logic,” *Theoretical Computer Science*, vol. 285, no. 2, pp. 187–243, 2002.
- [18] C. Olarte, E. Pimentel, and C. Rocha, “A rewriting logic approach to specification, proof-search, and meta-proofs in sequent systems,” *Journal of Logical and Algebraic Methods in Programming*, vol. 130, p. 100827, 2023.
- [19] J. Meseguer, “Twenty years of rewriting logic,” *The Journal of Logic and Algebraic Programming*, vol. 81, no. 7-8, pp. 721–781, 2012.
- [20] S. Escobar, C. Meadows, and J. Meseguer, “Maude-npa: Cryptographic protocol analysis modulo equational properties,” in *International School on Foundations of Security Analysis and Design*. Springer, 2007, pp. 1–50.
- [21] M. Souad, B. Faiza, and H. Nabil, “Formal modeling iot systems on the basis of biagents* and maude,” in *2020 International Conference on Advanced Aspects of Software Engineering (ICAASE)*. IEEE, 2020, pp. 1–7.
- [22] S. Eker, J. Meseguer, and A. Sridharanarayanan, “The maude ltl model checker,” *Electronic Notes in Theoretical Computer Science*, vol. 71, pp. 162–187, 2004.
- [23] Z. H. Toman, L. Hamel, S. H. Toman, M. Graiet, and D. C. G. Valadares, “Formal verification for security and attacks in iot physical layer,” *Journal of Reliable Intelligent Environments*, vol. 10, no. 1, pp. 73–91, 2024.
- [24] A. Armando, R. Carbone, and L. Compagna, “Ltl model checking for security protocols,” *Journal of Applied Non-Classical Logics*, vol. 19, no. 4, pp. 403–429, 2009.

Appendix

```
fmod LTL is
protecting BOOL .
sorts Formula .

*** primitive LTL operators
ops True False : -> Formula [ctor format (g o)] .
op ~_ : Formula -> Formula [ctor prec 53 format (r o d)] .
op _/\_ : Formula Formula -> Formula
  [comm ctor gather (E e) prec 55 format (d r o d)] .
op _\/_ : Formula Formula -> Formula
  [comm ctor gather (E e) prec 59 format (d r o d)] .
op O_ : Formula -> Formula [ctor prec 53 format (r o d)] .
op _U_ : Formula Formula -> Formula
  [ctor prec 63 format (d r o d)] .
op _R_ : Formula Formula -> Formula
  [ctor prec 63 format (d r o d)] .

*** defined LTL operators
op _->_ : Formula Formula -> Formula
  [gather (e E) prec 65 format (d r o d)] .
op _<->_ : Formula Formula -> Formula [prec 65 format (d r o d)] .
op <>_ : Formula -> Formula [prec 53 format (r o d)] .
op []_ : Formula -> Formula [prec 53 format (r d o d)] .
op _W_ : Formula Formula -> Formula [prec 63 format (d r o d)] .
op _|->_ : Formula Formula -> Formula [prec 63 format (d r o d)] .
op _=>_ : Formula Formula -> Formula
  [gather (e E) prec 65 format (d r o d)] .
op _<=>_ : Formula Formula -> Formula [prec 65 format (d r o d)] .

vars f g : Formula .

eq f -> g = ~ f \/_ g .
eq f <-> g = (f -> g) /\ (g -> f) .
eq <> f = True U f .
eq [] f = False R f .
eq f W g = (f U g) \/_ [] f .
eq f |-> g = [] (f -> (<> g)) .
eq f => g = [] (f -> g) .
eq f <=> g = [] (f <-> g) .

*** negative normal form
eq ~ True = False .
eq ~ False = True .
eq ~ ~ f = f .
eq ~ (f \/_ g) = ~ f /\ ~ g .
eq ~ (f /\ g) = ~ f \/_ ~ g .
eq ~ O f = O ~ f .
eq ~ (f U g) = (~ f) R (~ g) .
eq ~ (f R g) = (~ f) U (~ g) .
endfm
```

Figure 4: LTL syntax in Maude following functional LTL module